AMCJワークショップ
共鳴するコード、SuperColliderで創る音の世界
- 実践とその先へ

2026年3月14日

田所淳

01: Sin波から始めるSuperColliderの基本

はじめに：サーバーを起動する

SuperColliderはクライアント（言語）とサーバー（音声処理）に分かれています。
音を鳴らす前に、まずオーディオサーバーを起動します。

```
s.boot;
```

- 起動後、Post windowに `Server is running` と表示される
- **コードの実行：** `Ctrl+Enter` / 選択範囲なら `Shift+Enter`
- **全音を止める：** `Ctrl+.` （ピリオド）

1. 最初の一音 — `SinOsc.ar()`

SuperColliderで音を鳴らす最もシンプルなコード。

```
{ SinOsc.ar() }.play
```

- `{ }` の中に音を生成する **UGen**（Unit Generator）を書く
- `.play` でサーバーへ送信・再生
- `SinOsc.ar()` はデフォルトで **440Hz**（A4）のサイン波を生成
- `.ar` = **Audio Rate**（オーディオレート）で動作

2. ステレオ再生

`SinOsc.ar()` だけでは左チャンネルのみに出力されます。

```
{ SinOsc.ar().dup(2) }.play    // 信号を2チャンネルに複製
{ SinOsc.ar() ! 2 }.play       // dup(2) の省略記法

{ SinOsc.ar().dup(2) }.scope   // オシロスコープで波形確認
```

メソッド	意味
`.dup(2)`	信号を2チャンネルに複製（ステレオ化）
`! 2`	`.dup(2)` の省略記法
`.scope`	オシロスコープで波形を表示

3. 音量の変更

`SinOsc.ar(freq, phase, mul, add)` の第3引数 `mul` で音量を設定します。

```
{ SinOsc.ar(440, 0.0, 0.3).dup(2) }.play  // 音量 0.3

// キーワード引数で書くと明瞭
{ SinOsc.ar(freq: 440, mul: 0.3).dup(2) }.play
```

- `mul` の適切な範囲は **0.0 〜 1.0**
- 1.0 を超えるとクリッピング（音割れ）が発生する

4. 周波数の変更

第1引数 `freq` で音の高さを変えます。周波数を2倍にすると1オクターブ上になります。

```
{ SinOsc.ar(220).dup(2) }.play    // A3 (220Hz)
{ SinOsc.ar(440).dup(2) }.play    // A4 (440Hz) 標準ピッチ
{ SinOsc.ar(880).dup(2) }.play    // A5 (880Hz) 1オクターブ上
```

MIDIノートナンバーから周波数に変換することもできます：

```
69.midicps.postln                        // → 440.0 (A4)
{ SinOsc.ar(69.midicps).dup(2) }.play
```

5. 変数による指定

値に名前をつけて管理することで、コードが読みやすくなります。

```
// ローカル変数（var）：括弧の中でのみ有効
(
var freq = 440, amp = 0.3;
{ SinOsc.ar(freq, mul: amp).dup(2) }.play;
)

// 環境変数（~）：どこからでもアクセス可能
~freq = 330;
~amp  = 0.3;
{ SinOsc.ar(~freq, mul: ~amp).dup(2) }.play
```

6. コントロール信号でパラメータを変化させる

時間とともに変化するUGen（`.kr`）でパラメータを動かす。

```
// ビブラート：5Hzのサイン波で周波数を 400〜480Hz で揺らす
(
{ var freq;
  freq = SinOsc.kr(5).range(400, 480);  // .range で範囲を指定
  SinOsc.ar(freq, mul: 0.3).dup(2);
}.play )
```

UGen	説明
`SinOsc.kr(rate)`	正弦波のLFO
`LFNoise1.kr(rate)`	ランダムな直線補間信号
`XLine.kr(start, end, dur)`	指数的に変化する信号

6b. コントロール信号の応用

```
// トレモロ：4Hzのサイン波で音量を 0〜0.5 で揺らす
(
{ var amp;
  amp = SinOsc.kr(4).range(0, 0.5);
  SinOsc.ar(440, mul: amp).dup(2);
}.play )

// ランダムに周波数が変化
(
{ var freq;
  freq = LFNoise1.kr(2).range(200, 800);
  SinOsc.ar(freq, mul: 0.3).dup(2);
}.play )
```

7. 音を混ぜる

複数のUGenを `+` **で加算** するだけでミックスできます。

```
(
{ var sig1, sig2, sig3;
  sig1 = SinOsc.ar(220, mul: 0.2).dup(2);   // 基音
  sig2 = SinOsc.ar(440, mul: 0.2).dup(2);   // 第2倍音
  sig3 = SinOsc.ar(660, mul: 0.2).dup(2);   // 第3倍音
  (sig1 + sig2 + sig3) / 3.0;               // 加算でミックス
}.play )
```

8. エンベロープとは

エンベロープ（Envelope）とは音量の時間的な変化の形です。

```
音量
 1 |    *
   |   * *
 S |      *****
   |           *
 0 |___A__D__S__R___→ 時間
```

要素	意味
A ttack	発音〜最大音量までの時間
D ecay	最大音量〜サステインまでの時間
S ustain	キーを押している間の音量レベル
R elease	キーを離してから消えるまでの時間

8b. Env.perc — パーカッシブなエンベロープ

打楽器のような「アタック → フェードアウト」の形。

```
(
{ var sig, env;
  // Env.perc(attackTime, releaseTime)
  env = EnvGen.kr(Env.perc(0.01, 1.5), doneAction: Done.freeSelf);
  sig = SinOsc.ar(440);
  (sig * env).dup(2);  // 音源にエンベロープを掛ける
}.play )
```

- `EnvGen.kr` でエンベロープ信号を生成
- `doneAction: Done.freeSelf` でエンベロープ終了後に自動解放

8c. Env.adsr — ADSRエンベロープ

`gate` 引数でキーオン/オフを制御します。

```
(
f = { arg gate = 1;
  var sig, env;
  env = Env.adsr(0.1, 0.3, 0.5, 1.5);  // A, D, S, R
  env = EnvGen.kr(env, gate, doneAction: Done.freeSelf);
  sig = SinOsc.ar(440);
  (sig * env * 0.5).dup(2);
}; )

x = f.play;        // 再生（gate=1でアタック開始）
x.set(\gate, 0);   // リリース開始
```

9. SynthDef — 楽器を定義する

SynthDef はサーバーに「楽器の設計図」を登録する仕組みです。

```
// 定義
(
SynthDef("SinNote", {
  arg out = 0, freq = 440, amp = 0.3, gate = 1;
  var sig, env;
  env = EnvGen.kr(Env.adsr(0.01, 0.2, 0.7, 1.0), gate, doneAction: 2);
  sig = SinOsc.ar(freq) * env * amp;
  Out.ar(out, sig.dup(2));
}).add; )
```

9. SynthDef — 楽器を定義する

```
// 演奏
a = Synth("SinNote", [\freq: 261.63]);  // C4
b = Synth("SinNote", [\freq: 329.63]);  // E4
c = Synth("SinNote", [\freq: 392.00]);  // G4
a.set(\gate, 0); b.set(\gate, 0); c.set(\gate, 0);
```

10. 加算合成（Additive Synthesis）

多数のサイン波を重ねて複雑な音色を作る手法。

倍音列： 基音 f, 2f, 3f, 4f, 5f … 各倍音の振幅バランスが音色を決定

```
(
SynthDef("AddSynth", { arg out=0, freq=110, amp=0.3, gate=1;
  var sig = 0, env;
  sig = sig + SinOsc.ar(freq * 1, mul: 1.0/1);  // 基音
  sig = sig + SinOsc.ar(freq * 2, mul: 1.0/2);  // 第2倍音
  sig = sig + SinOsc.ar(freq * 3, mul: 1.0/3);  // 第3倍音
  sig = sig + SinOsc.ar(freq * 4, mul: 1.0/4);  // 第4倍音
  env = EnvGen.kr(Env.asr(0.01, 1.0, 1.5), gate, doneAction: 2);
  Out.ar(out, sig.dup(2) * env * amp * 0.2);
}).add; )
```

10b. Klang — 効率的な加算合成

Klang は多数のサイン波を一度に生成できる専用UGenです。

```
(
SynthDef("KlangSynth", { arg out=0, amp=0.5, gate=1;
  var num = 30, freqs, amps, phases, sig, env;
  freqs  = Array.exprand(num, 80, 12000).sort;
  amps   = Array.exprand(num, 0.002, 0.05).sort.reverse;
  phases = Array.rand(num, 0, 2pi);
  sig = Klang.ar(`[freqs, amps, phases]);  // バッククォートに注意
  env = EnvGen.kr(Env.asr(1.0, 1.0, 3.0), gate, doneAction: 2);
  Out.ar(out, sig.dup(2) * env * amp);
}).add; )
x = Synth("KlangSynth"); x.set(\gate, 0);
```

10c. DynKlang — 動的に変化する加算合成

DynKlang は各倍音をリアルタイムで変調できる動的版です。

```
(
SynthDef("DynKlangSynth", { arg out=0, amp=0.4, gate=1;
  var num=30, freqs, amps, phases, sig, env;
  freqs  = Array.exprand(num, 80, 8000).sort;
  amps   = Array.exprand(num, 0.005, 0.1).sort.reverse;
  phases = Array.rand(num, 0, 2pi);
  sig = DynKlang.ar(`[
    freqs * LFNoise1.kr(0.05 ! num).exprange(0.5, 2.0),  // 周波数変調
    amps  * LFNoise1.kr(0.5  ! num).exprange(0.1, 1.0),  // 振幅変調
    phases]);
  env = EnvGen.kr(Env.asr(3.0, 1.0, 5.0), gate, doneAction: 2);
  Out.ar(out, sig.dup(2) * env * amp);
}).add; )
```

11. 変調合成とは

モジュレーターの出力で**キャリア**のパラメータを変化させる手法。

```
              モジュレーター
                   ↓
キャリア → [パラメータ変調] → 出力
```

種類	変調するパラメータ	特徴
AM（振幅変調）	振幅	キャリア ± モジュレーターの側波帯
RM（リング変調）	振幅（双極性）	キャリア成分が消え側波帯のみ
FM（周波数変調）	周波数	豊かな倍音、少ないUGenで複雑な音色

11b. AM変調（振幅変調）

モジュレーターを **0〜1** にスケールしてキャリアの振幅に掛ける。

```
(
SynthDef("AmSynth", { arg out=0, gate=1, freq=440, modFreq=100, amp=0.5;
  var sig, env, mod;
  mod = SinOsc.ar(modFreq).range(0, 1);  // 0〜1 にマッピング（AM）
  sig = SinOsc.ar(freq, mul: mod);       // キャリアの振幅を変調
  env = EnvGen.kr(Env.perc(0.01, 1.5), gate, doneAction: 2);
  Out.ar(out, sig.dup(2) * env * amp);
}).add; )

Synth("AmSynth", [\freq: 440, \modFreq: 20]);   // うなり音
Synth("AmSynth", [\freq: 440, \modFreq: 440]);  // 金属的
```

11c. RM変調（リング変調）

モジュレーターを **-1〜1** のままキャリアの振幅に掛ける。
キャリア成分が消え、**上下の側波帯のみ**が残ります。

```
(
SynthDef("RmSynth", { arg out=0, gate=1, freq=440, modFreq=100, amp=0.5;
  var sig, env, mod;
  mod = SinOsc.ar(modFreq);          // -1〜1 そのまま使う（RM）
  sig = SinOsc.ar(freq, mul: mod);
  env = EnvGen.kr(Env.perc(0.01, 1.5), gate, doneAction: 2);
  Out.ar(out, sig.dup(2) * env * amp);
}).add; )

// AMとRMの違いを聴き比べる（キャリア成分の有無）
Synth("AmSynth", [\freq: 440, \modFreq: 110]);
Synth("RmSynth", [\freq: 440, \modFreq: 110]);
```

11d. FM変調（周波数変調）

モジュレーターをキャリアの**周波数**に加算する。
少ないUGenで豊かな倍音を持つ複雑な音色が作れます。

```
(
SynthDef("FmSynth", { arg out=0, carFreq=440, ratio=1.0, index=3, dur=1.0, amp=0.4;
  var mod, car, modFreq;
  modFreq = carFreq * ratio;                          // C:M比で倍音構成が決まる
  mod = SinOsc.ar(modFreq, mul: index * modFreq); // 変調深度 = index × modFreq
  car = SinOsc.ar(carFreq + mod);                     // 周波数にモジュレーターを加算
  Out.ar(out, (car * EnvGen.kr(Env.perc(0.01, dur), doneAction:2) * amp).dup(2));
}).add; )
```

- **C:M比 整数比** → 調和的な音色（音楽的）
- **C:M比 非整数比** → 非調和音（ベル、金属的）
- **index 大** → 倍音豊か（明るい音）

11e. FM変調 — C:M比とインデックスの効果

```
// C:M比による音色の変化
Synth("FmSynth", [\carFreq: 220, \ratio: 1.0,   \index: 5, \dur: 2]);  // ユニゾン
Synth("FmSynth", [\carFreq: 220, \ratio: 1.5,   \index: 5, \dur: 2]);  // 3:2（5度）
Synth("FmSynth", [\carFreq: 220, \ratio: 1.414, \index: 5, \dur: 2]);  // √2（ベル）

// インデックスによる音色の変化（ratio=1.0 固定）
Synth("FmSynth", [\carFreq: 330, \ratio: 1.0, \index: 1,  \dur: 3]);  // 純音に近い
Synth("FmSynth", [\carFreq: 330, \ratio: 1.0, \index: 5,  \dur: 3]);  // やや明るい
Synth("FmSynth", [\carFreq: 330, \ratio: 1.0, \index: 20, \dur: 3]);  // 非常に明るい
```

12. 本格的なFMシンセ — インデックスのエンベロープ制御

変調インデックス自体をエンベロープで変化させると、
発音直後は倍音豊か → 時間とともに柔らかく なる自然な音色変化が得られます。

```
(
SynthDef("FmExpressive", { arg out=0, gate=1, carFreq=440, ratio=1.414, index=10, rel=3.0, amp=0.4;
  var modFreq, idxEnv, mod, car, sig, env;
  modFreq = carFreq * ratio;
  // インデックスを発音後に素早く下げる（音色エンベロープ）
  idxEnv  = EnvGen.kr(Env.perc(0.001, rel * 1.5), gate).range(0, index);
  mod = SinOsc.ar(modFreq, mul: modFreq * idxEnv);
  car = SinOsc.ar(carFreq + mod);
  env = EnvGen.kr(Env.adsr(0.01, 0.3, 0.6, rel), gate, doneAction: 2);
  sig = GVerb.ar((car * env * amp).dup(2), 30, 2.0) * 0.8;
  Out.ar(out, sig);
}).add; )
x = Synth("FmExpressive", [\carFreq: 440, \rel: 4.0]); x.set(\gate, 0);
```

12b. SuperSaw シンセサイザー

複数のノコギリ波を**デチューン**（音程をわずかにずらす）して重ねることで、
EDM/トランスで定番の分厚いサウンドが得られます。

```
(
SynthDef("SuperSaw", { arg out=0, gate=1, freq=440, detune=0.15, cutoff=3000, rel=1.0, amp=0.3;
  var sig, env, voices = 5;
  sig = Mix.fill(voices, { |i|
    var d = detune * (i - (voices-1)/2) / (voices-1);
    Saw.ar(freq * (1 + d));                // 各ボイスを均等にデチューン
  });
  sig = RLPF.ar(sig, cutoff, 0.3);       // ローパスフィルターで整える
  env = EnvGen.kr(Env.adsr(0.01, 0.2, 0.7, rel), gate, doneAction: 2);
  sig = GVerb.ar(sig.dup(2), 30, 1.5) * env * amp * (1/voices);
  Out.ar(out, sig);
}).add; )
```

12c. SuperSaw — コードで演奏

```
// Cメジャーコード（C4, E4, G4）
(
var freqs = [261.63, 329.63, 392.00];
~pad = freqs.collect { |f|
  Synth("SuperSaw", [\freq: f, \detune: 0.2, \cutoff: 2500, \amp: 0.25]);
}; )

~pad.do { |s| s.set(\gate, 0) };  // 停止

// パッドとして（長いアタック・リリース）
(
~pad = [220, 277.18, 329.63].collect { |f|
  Synth("SuperSaw", [\freq: f, \detune: 0.25, \attk: 2.0, \rel: 4.0,
                     \cutoff: 1500, \amp: 0.2]);
}; )
~pad.do { |s| s.set(\gate, 0) };
```

まとめ

テーマ	キーワード
最初の一音	`SinOsc.ar()`, `.play`, `.dup(2)`
音量・周波数	`mul`, `freq`, `.midicps`
変数	`var`, `~name`
コントロール信号	`.kr`, `SinOsc.kr`, `LFNoise1`, `.range`
音を混ぜる	`+`, `Mix`
エンベロープ	`Env.perc`, `Env.adsr`, `EnvGen.kr`
楽器定義	`SynthDef`, `Synth`, `Out.ar`
加算合成	倍音列, `Klang`, `DynKlang`
変調合成	AM / RM / FM, C:M比, インデックス
応用	FMシンセ, SuperSaw